PORTFOLIO PROJECT 4 · DETAILED REQUIREMENTS · V1.0 + KUBERNETES

Durable, Distributed Runtime for LangGraph Agents

Numbered, testable requirements. **MUST** = required for v1.0; **SHOULD** = include if time allows. Every MUST maps to an acceptance criterion in §11. Section 10 (new) runs the whole system on Kubernetes, written so each requirement teaches one concept.

v1.0 · Python 3.12 · LangGraph 1.x · FastAPI · Redis 7 (Streams + Pub/Sub) · Postgres 16 + pgvector · OpenTelemetry · React 18 + TS · Docker (multi-stage) · Kubernetes · kind · Helm · KEDA · Apache-2.0

1 Runtime core

A *run* is one durable execution of a compiled LangGraph against an input. The runtime owns its lifecycle; the graph owns its logic.

RUN STATE MACHINE

`queued` → `running` → `waiting_approval` → `running` → `completed`

· running → `waiting_children` → running · running → queued (lease expired or released) · any → `failed` / `cancelled`

R1.1 Run identity & lifecycle MUST

Each run has a UUIDv7 `run_id` (time-ordered), a `graph_name`, an `input` JSON, a status from the state machine above, and a monotonically increasing `epoch` (see R2.3). Status transitions are written to Postgres in a single transaction with whatever caused them.

R1.2 Checkpointing MUST

Use LangGraph's Postgres checkpointer with `thread_id = run_id`. A checkpoint is written after every super-step (node completion). Checkpoints are the source of truth for resume; the `runs` table is the status index, never the state. SQLite checkpointer is not supported.

R1.3 Resume MUST

A run can be resumed from its latest checkpoint by id when its worker died (reclaim), its worker shut down gracefully (R10.7), it was waiting for approval and was approved, or it was explicitly paused. Resume is idempotent: resuming a run that is already `running` returns its current status and does nothing.

R1.4 Human-in-the-loop interrupt MUST

When a node calls `interrupt(payload)`, the runtime persists the payload on the run, sets status `waiting_approval`, emits `interrupt.raised`, releases the lease, and acks the queue message. The run must be approvable with **no live worker** — approval re-enqueues it. Approval body is `{decision: approve|reject, edits?: object}`; on approve, `edits` is passed as the interrupt's resume value; on reject the run becomes `cancelled` with `reason=rejected`.

R1.5 Event stream MUST

Every run emits typed events `{run_id, seq, ts, epoch, type, payload}` where `seq` is strictly increasing per run. Events are **persisted** (append-only `run_events`) *before* being published to pub/sub, so late or reconnecting subscribers can replay via `after_seq`. Types:

- `run.started · run.completed · run.failed · run.cancelled · run.reclaimed · run.released`
- `node.started · node.completed · node.failed` (payload: node name, duration, tokens, cost)
- `tool.called · tool.completed` (payload: tool, args summary, duration, `cached: bool`)
- `token` (payload: node, delta) — pub/sub only, not persisted, to keep the table small
- `interrupt.raised · child.started · child.completed`

R1.6 **Cancellation** MUST

Cooperative: `POST /runs/{id}/cancel` sets a cancel flag; the worker checks it between nodes and before each tool call, then writes `cancelled` and releases the lease. A cancelled run cannot be resumed.

R1.7 **Run limits** MUST

Per-run wall-clock timeout (default 20 min) and max super-steps (default 200). Exceeding either → `failed` with `reason=timeout|step_limit`. Configurable per run via `options`.

---

2 Distributed execution

Stateless API replicas, a work stream, a worker pool, and Postgres as the only source of truth. Redis holds nothing that can't be rebuilt. Everything in this section is platform-agnostic; §10 maps it onto Kubernetes.

R2.1 **Work queue** MUST

A Redis Stream `runs:work` with a consumer group `workers`. A message carries only `{run_id}` — state lives in Postgres. Delivery is at-least-once: a worker `XACK`s only after the run reaches a terminal state, an interrupt, `waiting_children`, or a graceful release. Unacked messages idle longer than the lease TTL are reclaimable via `XAUTOCLAIM`. The stream's pending-entries count is the autoscaling signal in R10.9.

R2.2 **Worker pool** MUST

`runtime worker --concurrency C` runs one worker process with up to C concurrent runs; each process joins the consumer group as `{hostname}:{pid}` (on Kubernetes the hostname is the pod name, so ownership is traceable to a pod). Scale-out is by running more processes: `--workers N` locally and in Compose, **pod replicas** on Kubernetes. A worker's loop per run: claim message → acquire lease → load checkpoint → execute until a stopping condition → persist → release lease → ack. Workers are stateless and safe to kill at any instant.

R2.3 **Leases with fencing** MUST

Before executing, a worker acquires the run's lease:

```
UPDATE runs SET owner=:me, epoch=epoch+1, lease_expires_at=now()+:ttl
 WHERE id=:run_id AND (lease_expires_at < now() OR owner IS NULL)
RETURNING epoch;
```

The returned `epoch` is a **fencing token**: every write the worker makes (checkpoint metadata, events, tool results, findings) carries it, and the database rejects writes whose epoch is lower than the run's current epoch. A heartbeat extends the lease every `ttl/3`. Defaults: ttl 30 s, heartbeat 10 s. This is what makes a paused-then-resumed "zombie" worker harmless.

R2.4 **Reclaim** MUST

A reaper loop in every worker (every 5 s) finds runs with status `running` and an expired lease, acquires the lease (bumping epoch), emits `run.reclaimed`, and resumes from the last checkpoint. After `max_reclaims` (default 3) the run is `failed` with `reason=max_reclaims`. Reclaim never re-executes a completed super-step. A *graceful* release (R10.7) does not count toward `max_reclaims`.

R2.5 **Streaming across replicas** MUST

Workers publish each event to Redis Pub/Sub channel `run:{run_id}` after persisting it. An API replica serving a stream subscribes to the channel first, then replays `run_events` from `after_seq`, then relays live events, **deduplicating by** `seq`. If a live event arrives with a gap (`seq > last+1`), the replica back-fills the gap from Postgres before forwarding. Because any replica can serve any stream, the load balancer needs **no session affinity**.

R2.6 **Idempotent tool calls** MUST

Every tool call is wrapped. Its key is `sha256(run_id · node · call_index · tool · canonical_json(args))` — deliberately *excluding* epoch so a retry after reclaim hits the same key. Table `tool_invocations(key PK, run_id, tool, status, result, epoch, created_at)`. Key exists with `status=done` → return stored result, emit `tool.completed{cached:true}`, do not execute. Key exists with `status=in_progress` and a *lower* epoch → treat as abandoned, re-execute under the new epoch. Tools declare `side_effect: bool`; side-effecting tools additionally use the key as their own primary key with `ON CONFLICT DO NOTHING`.

R2.7 **Trace propagation** MUST

The API starts an OpenTelemetry trace on `POST /runs` and stores the W3C `traceparent` on the run. Workers and child runs continue that trace. Spans: `run` → `node:{name}` → `llm.call` / `tool:{name}`, with attributes for model, tokens, cost, cached, epoch — and, on Kubernetes, `k8s.pod.name` and `k8s.node.name`.

R2.8 **Sub-runs (distributed fan-out)** MUST

A node may spawn **child runs**: each child is a full run (own `run_id`, `parent_run_id`, own checkpoint thread) enqueued on `runs:work`, so children land on any worker pod. The parent transitions to `waiting_children`, releases its lease, and acks. Postgres tracks `child_total` / `child_done` atomically; when they match, the parent is re-enqueued and its fan-in node reads child results from `run_results`. If at least one child completed, the parent continues with `partial=true`; if all failed, the parent fails. `--local-fanout` runs children in-process (LangGraph `Send`) for tests.

R2.9 **Backpressure** MUST

Per-worker concurrency C bounds in-flight runs. Per-run `max_children` (default 6). Queue depth is exposed as a metric, and the API returns `429` with `Retry-After` when depth exceeds a configurable threshold — the safety valve for when autoscaling (R10.9) hits its max.

DESIGN DECISION — WHY SUB-RUNS INSTEAD OF LANGGRAPH `SEND`

`Send` fans out inside one super-step on one worker; it cannot cross a process boundary. Modelling each researcher as a child run is what lets the fan-out actually distribute across pods, and it gives every child its own checkpoint, lease, and reclaim for free. The cost is a parent/child protocol (R2.8). Document this trade-off in `docs/design.md`.

3 Pluggable interfaces

The repo must run standalone. These seams are where projects 1–3 attach later, and where contributors swap providers.

R3.1 **`ModelProvider`** MUST

Protocol: `complete(messages, tools?, response_format?, stream) → async iterator[Chunk]` plus usage (tokens, cost). Implementations: **OpenAI** (default) and **Ollama** (keyless local). Model per role is configurable: `MODEL_PLANNER`, `MODEL_RESEARCHER`, `MODEL_CRITIC`, `MODEL_SYNTHESIZER`. Selected by `MODEL_PROVIDER=openai|ollama`.

R3.2 **`ToolPolicy`** MUST

Protocol: `check(ctx, tool, args) → Allow | Deny(reason) | RequireApproval(payload)`, evaluated inside the tool wrapper before idempotency lookup. Default: static allow-list from config. `Deny` raises a tool error the agent sees; `RequireApproval` triggers R1.4. Adapter point for the policy engine (P2).

R3.3 **`SearchTool`** MUST

Protocol: `search(query, k) → [{title, url, snippet, content}]`. Default **fixture** implementation serves `fixtures/search/*.json` keyed by normalised query with fuzzy fallback, so demos and evals are deterministic and keyless. Optional real provider behind `SEARCH_PROVIDER`.

R3.4 **Tool registry** MUST

Tools are declared with `name, json_schema, side_effect: bool, cache: bool`. Registration wraps the tool with policy (R3.2), idempotency (R2.6), tracing (R2.7) and event emission automatically — agent code never sees these concerns.

---

4 Example agent — `examples/research`

Planner → Approval → Researcher×N → Critic → Synthesizer. Its shape exists to exercise the runtime: a pause, a distributed fan-out, a loop, and a side effect.

STATE SCHEMA

```
class ResearchState(TypedDict):
    question: str
    plan: Plan | None          # {version:int, sub_questions:[{id, text, done_criteria}]}
    findings: dict[str, list[Finding]]   # keyed by sub_question id
    critique: Critique | None  # {coverage:float, gaps:[str], contradictions:[str],
                               #  action:"synthesize"|"replan"|"research_more", targets:[id]}
    iteration: int
    answer: Answer | None      # {text:str, citations:[{n:int, finding_id, url, quote}]}

class Finding(TypedDict):
    id: str; sub_question_id: str; claim: str; evidence: str
    sources: list[{url, title, quote}]; confidence: float   # 0–1
```

R4.1 **Planner** MUST

Structured output (JSON schema enforced) → `Plan` with 3–6 sub-questions, each with explicit `done_criteria`. Then `interrupt({type:"plan_approval", plan})`. On resume, if `edits.plan` is present it replaces the plan with `version+1`. On a `replan` loop-back from the Critic, the Planner receives `critique.gaps` and may only *add* sub-questions; no second approval is required.

R4.2 **Researcher ×N** MUST

One child run per sub-question (R2.8), each a `create_agent` with tools `search`, `retrieve`, `save_finding`. Every claim cites ≥ 2 sources; max 8 tool calls per researcher; returns ≥ 1 `Finding` or an explicit `no_evidence` result. `save_finding(finding)` is the side-effecting tool: it upserts into `findings` keyed by the idempotency key, so a reclaimed researcher never writes a duplicate row.

R4.3 **Critic** MUST

Structured output → `Critique`. Routing is a **pure function** of the critique (unit-testable): `coverage ≥ 0.80` → synthesize; gaps naming topics not in the plan → `replan`; gaps naming existing sub-questions → `research_more` with `targets` (re-spawns children only for those ids); `iteration ≥ 3` → forced synthesize with a `limitations` note.

R4.4 **Synthesizer** MUST

Input is `findings` only. Output `Answer` with inline `[n]` citations mapped to finding sources. A post-hoc validator rejects any URL not present in `findings` (fail closed: strip the citation and flag `unverified_citation`). A forced synthesize adds a "Limitations" section listing unanswered sub-questions.

R4.5 **Retrieval** MUST

`runtime ingest ./docs` chunks files (markdown, txt, pdf), embeds them into pgvector tables `documents` / `chunks`. `retrieve(query, k=5)` returns chunks with document title and a citable source reference. On Kubernetes, ingest runs as a one-off Job (R10.6).

R4.6 **Fixtures** MUST

Ship ≥ 3 complete fixture sets (question + search fixtures + ingested docs) so the system runs a real end-to-end answer with no external keys. One fixture set is deliberately thin so the Critic loop is exercised.

5 API surface

ENDPOINT	PURPOSE	NOTES
`POST /runs`	Create a run	Body `{graph, input, options?}` → `{run_id}`. Starts the trace; enqueues. `429` under backpressure.
`GET /runs/{id}`	Status	Status, epoch, owner (pod name), lease expiry, cost, child summary, interrupt payload.
`GET /runs/{id}/events?after_seq=`	Replay	Persisted events, paged. Used for catch-up and reconnect.
`GET /runs/{id}/stream`	SSE	Backlog then live (R2.5). Heartbeat comment every 15 s.
`WS /ws/runs/{id}`	WebSocket	Same event stream for the UI; client sends `{after_seq}` on connect.
`POST /runs/{id}/approve`	HITL	`{decision, edits?}` (R1.4). 409 if not `waiting_approval`.
`POST /runs/{id}/resume`	Resume	Idempotent (R1.3). 409 if terminal.
`POST /runs/{id}/cancel`	Cancel	Cooperative (R1.6).
`GET /runs/{id}/result`	Result	Final answer. 404 until completed.
`GET /graphs · /graphs/{name}`	Introspection	Nodes and edges for the UI's React Flow view.
`GET /healthz`	Liveness	Process is up. No dependency checks (R10.5).
`GET /readyz`	Readiness	Postgres + Redis reachable and not draining. Fails during shutdown (R10.7).
`GET /metrics`	Ops	Prometheus metrics (R7.2).

All endpoints are stateless with respect to the replica; any pod can serve any run. OpenAPI schema is published and the UI's TypeScript client is generated from it.

6 UI — React + TypeScript

This is the artifact that proves front-end, so it is held to the same standard as the backend.

R6.1 **Live run view** MUST

React Flow graph built from `GET /graphs/{name}`. Node status is derived by a pure reducer `events → graphState` (unit-tested): `idle · running · completed · failed · waiting`, with the active edge animated. The Researcher node expands into N child chips with their own status and the **pod that ran them**. Side panels: token stream; tool-call feed (tool, args summary, duration, `cached` badge); header with status, elapsed, cost, epoch. WebSocket hook with automatic reconnect that resumes from the last `seq` — this is what makes API rolling updates invisible (AC15).

R6.2 **Approval screen** MUST

Opens on `interrupt.raised{type:"plan_approval"}`. Editable sub-question list (add, remove, edit text and done-criteria). Approve sends `edits` only if something changed. Reject prompts for a reason. Works after a page refresh (state rebuilt from `GET /runs/{id}`).

R6.3 **Answer view** MUST
Markdown render of the answer; `[n]` citations open a popover with title, URL, and the quoted evidence. "Limitations" renders when present.

R6.4 **Start form** MUST
Question input, graph selector, fixture-set selector, optional `max_children`. Submits `POST /runs` and navigates to the live view.

R6.5 **Engineering standard** MUST
React 18, TypeScript `strict`, Vite, React Flow, TanStack Query, Tailwind. Generated API client. Vitest + Testing Library covering the event reducer, the approval editor, and reconnect/resume. No `any`. Lighthouse accessibility ≥ 90. Served as static files from an nginx container (R10.1).

R6.6 **Fleet panel** SHOULD
Small panel showing queue depth, worker pod count, and which pod owns each active run — makes autoscaling (AC14) visible during the demo.

7 Observability & evaluation

R7.1 **Tracing** MUST
OpenTelemetry SDK → OTLP → **OpenTelemetry Collector** → **Jaeger**. One trace spans API → worker → child runs (R2.7). Structured JSON logs carry `run_id, epoch, trace_id` (and `pod` on Kubernetes) on every line.

R7.2 **Metrics** MUST
Prometheus at `/metrics`, Grafana dashboard checked into `ops/grafana/`: runs by status; queue depth and pending entries; active leases; reclaims vs graceful releases; tool cache hit ratio; LLM tokens and cost by model; node latency histograms; stream subscribers per replica; worker/API replica counts.

R7.3 **Eval suite** MUST
`evals/cases/*.yaml`: `{question, fixture_set, expected_claims[], expected_sources[], forbidden_claims[]}`, ≥ 20 cases. Scorers: **claim coverage** (LLM judge, fixed rubric, pinned judge model), **citation accuracy** (deterministic), **forbidden-claim rate**, p95 latency, cost. `make eval` writes `evals/results/<sha>.json` and a markdown table.

R7.4 **CI quality gate** MUST
PR fails if claim coverage drops > 5 points, citation accuracy < 0.95, forbidden-claim rate > 0, or p95 latency rises > 30% vs `evals/baseline.json`. Baseline changes only via an explicit `make eval-accept` commit.

R7.5 **Judge calibration** SHOULD
≥ 30 human-labelled claim/finding pairs with reported judge agreement (Cohen's κ), published in the README.

8 Test requirements

UNIT (NO SERVICES)

- Lease: acquire, expiry, re-acquire bumps epoch, stale-epoch write rejected, graceful release.
- Idempotency wrapper: same key → no second execution; different args → new key; abandoned `in_progress` with lower epoch re-executes.
- Event `seq` strictly monotonic; replay + live dedup; gap back-fill.
- Critic routing across coverage/gap/iteration cases.
- Citation validator strips unknown URLs.
- SIGTERM handler: stops claiming, finishes super-step, releases lease.
- UI event reducer from recorded event logs.

INTEGRATION (COMPOSE, IN CI)

- Fixture run end-to-end → `completed` with a cited answer.
- **Crash:** `SIGKILL` the owning worker mid-research → completes elsewhere; one `run.reclaimed`; no duplicate `findings`.
- **Zombie:** `SIGSTOP` past lease TTL, reclaim, `SIGCONT` → its writes are rejected.
- **Replicas:** stream on the non-hosting replica receives every event in order.
- **Approval survives restart.**

CLUSTER (KIND, IN CI) — §10

- Helm install → smoke run completes.
- Pod delete, node drain, rolling upgrade, scale-up/down scenarios (AC11–AC16).

9 Storage, configuration & packaging

TABLES (POSTGRES, ALEMBIC MIGRATIONS)

TABLE	KEY COLUMNS
`runs`	id, graph_name, parent_run_id, status, epoch, owner, lease_expires_at, input, options, interrupt_payload, child_total, child_done, reclaims, releases, traceparent, cost, created_at, finished_at
`run_events`	run_id, seq, ts, epoch, type, payload — PK (run_id, seq)
`run_results`	run_id PK, output, partial
`tool_invocations`	key PK, run_id, tool, status, result, epoch, created_at
`findings`	key PK (= idempotency key), run_id, sub_question_id, claim, evidence, sources, confidence
`documents · chunks`	document id/title/path; chunk id, document_id, text, embedding (vector)
`checkpoints · checkpoint_writes`	owned by the LangGraph Postgres checkpointer

R9.1 **Configuration** MUST
`pydantic-settings`, read from environment only, with a committed `.env.example`. The same variables feed Compose (`.env`) and Kubernetes (ConfigMap + Secret, R10.4) — one config model, two delivery mechanisms. Defaults make the fixture demo work with **only** `OPENAI_API_KEY`, or none with `MODEL_PROVIDER=ollama`.

R9.2 **Compose & Make targets** MUST
Compose stays the **fast inner loop**: `postgres, redis, api (×2), worker (×2), ui`; profile `observability` adds `otel-collector, jaeger, prometheus, grafana`. Targets: `make up · demo · demo-crash · demo-replicas · eval · test · ingest`, plus the cluster targets in R10.12.

R9.3 **Code standard** MUST

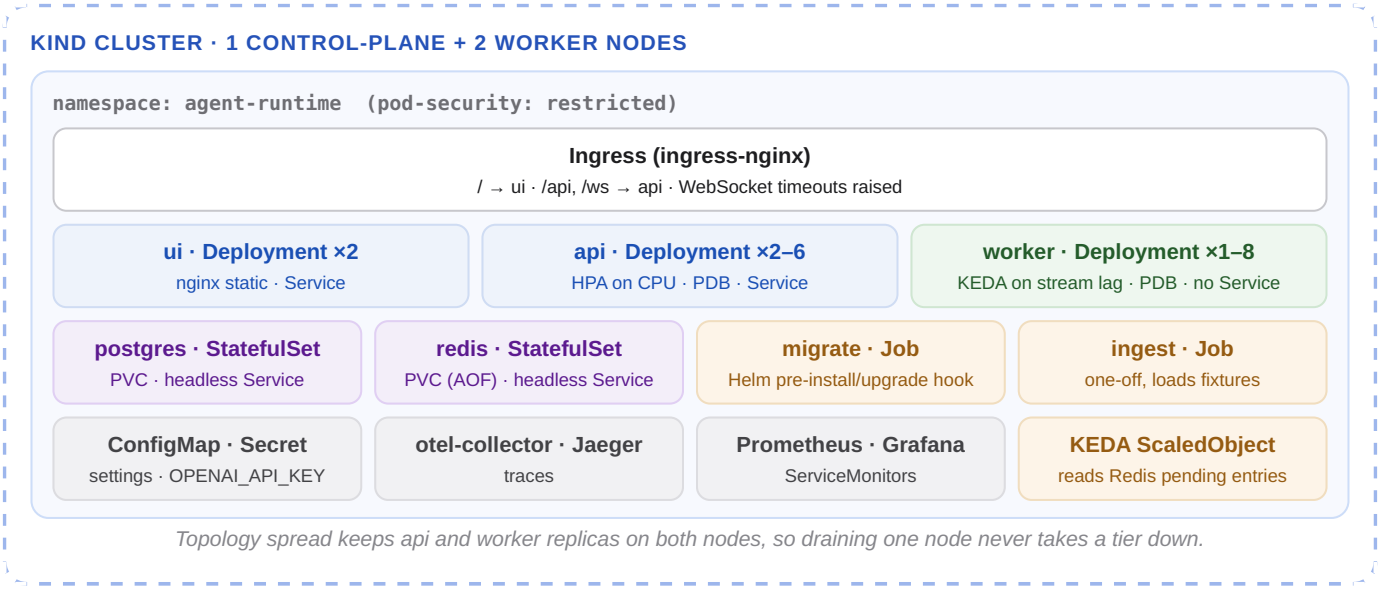
Python 3.12, `uv`, `ruff`, `mypy --strict` on `runtime`. Pinned dependencies; README names the LangGraph 1.x version targeted. Apache-2.0. Conventional commits; `CHANGELOG.md`; tag `v1.0.0`.

R9.4 **Security posture** MUST

README states plainly: tools execute with network access and no sandbox; run only against trusted documents and fixture data; no prod credentials. In-cluster hardening is R10.11.

10 Containers & Kubernetes NEW

The same system, deployed as it would be in production: container images, a local multi-node cluster, Helm, health probes, graceful shutdown, disruption budgets, and queue-driven autoscaling. Each requirement carries a **You learn** line naming the concept it teaches.



Topology spread keeps api and worker replicas on both nodes, so draining one node never takes a tier down.

IMAGES

R10.1 **Container images** MUST

Three images — `api`, `worker` (same Python base, different entrypoint), `ui` (Vite build → `nginx-unprivileged`). Requirements: **multi-stage** builds (builder with `uv` → slim runtime, no compilers); runs as a **non-root** UID; no secrets baked in; `.dockerignore`; each image < 300 MB (UI < 50 MB); tagged with the git SHA and semver; built in CI and pushed to **GHCR**; image vulnerability scan (Trivy) in CI failing on critical CVEs. SHOULD: multi-arch (amd64 + arm64) via `buildx`, and an SBOM.

You learn: layer caching, multi-stage builds, image size and attack surface, registries and immutable tags.

CLUSTER & WORKLOADS

R10.2 **Local cluster** MUST

`make k8s-up` creates a **kind** cluster from a checked-in config (1 control-plane + 2 worker nodes, port mappings for ingress), installs ingress-nginx, metrics-server and KEDA, and loads locally built images with `kind load docker-image` so no registry is needed for development. `make k8s-down` tears it down. Works on a 16 GB laptop.

You learn: what a node, control plane and kubelet are; how images get onto nodes; cluster add-ons.

R10.3 Workloads MUST

- **api**: Deployment (2 replicas), ClusterIP Service, routed by an Ingress. Ingress annotations raise proxy read/send timeouts to 3600 s for SSE/WebSocket and disable buffering for SSE. **No sticky sessions** — R2.5 makes them unnecessary, and the README says why.
- **worker**: Deployment (min 1), no Service (workers pull from Redis; nothing calls them). Pod name passed in via the Downward API as `WORKER_ID`.
- **ui**: Deployment (2 replicas) + Service.
- **postgres** (pgvector image) and **redis** (AOF on): StatefulSets with PersistentVolumeClaims and headless Services. README notes that production would use a managed database or an operator (e.g. CloudNativePG), and why stateful data in-cluster is a deliberate learning choice here.

You learn: Deployment vs StatefulSet, Services and DNS, Ingress, PVCs, the Downward API, and why stateless tiers scale freely.

R10.4 Config & secrets MUST

Non-secret settings in a **ConfigMap**; `OPENAI_API_KEY` and database passwords in a **Secret** created by `make k8s-secrets` from the local `.env` — never committed, never in Helm values. Pods consume both via `envFrom`. A config change triggers a rollout via a checksum annotation on the pod template. SHOULD: document External Secrets / Sealed Secrets as the production path.

You learn: ConfigMap vs Secret, why base64 isn't encryption, and how config changes roll pods.

R10.5 Health probes MUST

- **api**: liveness `/healthz` (process only — never checks dependencies, so a Postgres blip doesn't restart every pod); readiness `/readyz` (Postgres + Redis reachable, not draining); startup probe allowing 60 s.
- **worker**: a tiny HTTP server on `:8081` — liveness fails if the main loop hasn't ticked in 30 s (a hung event loop); readiness fails while draining.
- Probe periods and thresholds documented with the reasoning.

You learn: liveness vs readiness vs startup probes, and the classic mistake of putting dependency checks in liveness.

R10.6 Migrations & one-off jobs MUST

Alembic migrations run as a Kubernetes **Job** wired as a Helm `pre-install,pre-upgrade` hook, so schema is in place before any new api/worker pod starts; migrations must be backward-compatible with the previous release (expand/contract) so a rolling upgrade never runs old code against an incompatible schema. Fixture ingest (R4.5) runs as a separate Job via `make k8s-ingest`. Jobs have `backoffLimit` and `ttlSecondsAfterFinished`.

You learn: Jobs, Helm hooks, and zero-downtime schema changes.

R10.7 Graceful shutdown MUST

Kubernetes sends `SIGTERM`, waits `terminationGracePeriodSeconds`, then `SIGKILL`s. The runtime must use that window:

- **worker** (grace 60 s): on SIGTERM, flip readiness to failing, stop claiming new messages, let each in-flight run finish its *current super-step* (so a checkpoint is written), then **release the lease immediately** (`owner=NULL`, status `queued`), re-enqueue the run, emit `run.released`, and exit. Another pod picks it up in seconds instead of waiting for the 30 s lease TTL. If the step can't finish in time, the SIGKILL path (R2.4 reclaim) still covers it.
- **api** (grace 30 s): a `preStop` sleep of 5 s so the endpoint is removed from the Service before the pod stops accepting; on SIGTERM, fail `/readyz`, stop accepting new streams, send each open stream a `reconnect` event, then close. Clients reconnect to another pod with `after_seq` and lose nothing.

You learn: the pod termination lifecycle, preStop hooks, endpoint propagation delay — and how your lease design turns a shutdown into a sub-second hand-off.

R10.8 **Resources, placement & disruption** MUST

Every container sets CPU/memory **requests** and a memory **limit** (values documented with how they were measured). **PodDisruptionBudgets**: api `minAvailable: 1`, worker `maxUnavailable: 1`. **topologySpreadConstraints** across nodes for api and worker. Rolling update strategy `maxUnavailable: 0, maxSurge: 1` for api.

You learn: requests vs limits and QoS classes, OOMKills, scheduling, PDBs and voluntary disruption, rollout strategy.

SCALING

R10.9 **Autoscaling** MUST

- **worker**: a **KEDA ScaledObject** using the Redis Streams scaler on `runs:work` / group `workers`, targeting pending entries per pod ≈ `concurrency`. min 1, max 8, cooldown 120 s. Scale-down relies on R10.7 so a removed pod hands its runs off cleanly.
- **api**: a **HorizontalPodAutoscaler** on CPU (target 70%), min 2, max 6.
- README explains why workers scale on *queue lag* rather than CPU (LLM-bound workers sit idle on CPU while waiting on the model, so CPU is a misleading signal).

You learn: HPA mechanics, event-driven autoscaling with KEDA, choosing the right scaling signal, and scale-down safety.

PACKAGING, SECURITY & OPS

R10.10 **Helm chart** MUST

`deploy/helm/agent-runtime` templates every object above, with `values.yaml` (defaults), `values-kind.yaml` (local), and `values-cloud.example.yaml` (managed Postgres/Redis, real registry, TLS). Image tags, replica bounds, resources, probe timings, and scaler thresholds are all values. `helm lint` and **kubeconform** schema validation run in CI. `helm upgrade --install` is idempotent and `helm rollback` works.

You learn: templating, values layering, release history and rollback.

R10.11 **Pod security** MUST

Namespace labelled for the **restricted** Pod Security Standard. All app pods: `runAsNonRoot`, `readOnlyRootFilesystem` (with an `emptyDir` for `/tmp`), `allowPrivilegeEscalation: false`, all capabilities dropped, `seccompProfile: RuntimeDefault`, a dedicated ServiceAccount with `automountServiceAccountToken: false`. SHOULD: **NetworkPolicies** — default-deny, api reachable only from ingress, Postgres/Redis reachable only from api/worker/jobs, worker egress limited to Postgres, Redis, DNS and the LLM provider.

You learn: securityContext, Pod Security Standards, least-privilege service accounts, and network segmentation — all directly relevant to the security roles you're targeting.

R10.12 **Make targets & cluster observability** MUST

Targets: `make k8s-up · k8s-build · k8s-load · k8s-secrets · k8s-deploy · k8s-ingest · k8s-demo-podkill · k8s-demo-drain · k8s-demo-scale · k8s-demo-upgrade · k8s-down`. Each demo prints PASS/FAIL. The OTel Collector runs as a Deployment; Prometheus scrapes api/worker via ServiceMonitors (kube-prometheus-stack, or a minimal Prometheus in `values-kind.yaml`); the Grafana dashboard adds pod count, restarts, and KEDA desired-replicas panels.

You learn: operating a cluster — kubectl, logs, events, describe, and reading the system through metrics.

R10.13 **CI on a real cluster** MUST

A GitHub Actions job builds images, creates a kind cluster, `helm install`s the chart, runs a smoke run, then runs `k8s-demo-podkill` and `k8s-demo-upgrade`. On failure it uploads `kubectl get events`, pod logs and `describe` output as artifacts. The README shows the badge.

You learn: ephemeral test clusters and debugging failed rollouts from CI artifacts.

R10.14 `docs/kubernetes.md` **MUST**

A walkthrough of every object in the chart: what it is, why it exists in this system, and what breaks without it — plus the measured numbers (hand-off time on graceful shutdown vs crash, scale-up latency, rollout duration). This is the document that proves you understand the cluster rather than copied manifests, and it's the second-best thing to link in the post after the GIF.

R10.15 **Cloud deployment** **SHOULD**

One documented deploy to a managed cluster (AKS, EKS or GKE) using `values-cloud.example.yaml`, with managed Postgres and Redis, cert-manager TLS, and a teardown script so it doesn't run up a bill. A short write-up of what differed from kind.

KUBERNETES LEARNING PATH — BUILD §10 IN THIS ORDER

STEP	WHAT YOU DO	CONCEPTS	~TIME
1 · Containerize	Dockerfiles for api / worker / ui; run them with Compose.	Images, layers, multi-stage, non-root	4–6 h
2 · First deploy	kind cluster; raw YAML for Postgres, Redis, api, worker; port-forward and run one question.	Pods, Deployments, StatefulSets, Services, DNS, PVCs	5–7 h
3 · Make it healthy	ConfigMap/Secret, probes, migration Job, Ingress with WebSocket.	Config, probes, Jobs, Ingress	4–6 h
4 · Make it survive	Graceful shutdown, PDBs, topology spread; pod-kill and drain demos.	Termination lifecycle, disruption, scheduling	5–7 h
5 · Make it scale	HPA for api, KEDA for workers; scale demo.	Autoscaling, scaling signals	3–5 h
6 · Package & harden	Convert YAML to Helm; securityContext; NetworkPolicies; CI on kind.	Helm, Pod Security, NetworkPolicy, CI	6–9 h

Write raw YAML first and convert to Helm last — templating manifests you don't yet understand hides exactly what you're trying to learn. **§10 total: ~27–40 focused hours.** Whole project including §10: **~100–130 focused hours, ~10–13 weeks part-time.**

11 Acceptance criteria — v1.0 is done when all pass

CORE RUNTIME (COMPOSE)

- **AC1 · Quickstart.** Fresh clone, `docker compose up`, one fixture question → `completed` with a cited answer in < 5 min, with only `OPENAI_API_KEY` (or none via Ollama).
- **AC2 · Crash-and-resume.** `make demo-crash`: owning worker `SIGKILL`ed mid-research; run completes on the other worker; exactly one `run.reclaimed`; epoch incremented; zero duplicate `findings`; no re-executed completed super-step.
- **AC3 · Stream from any replica.** `make demo-replicas`: every event arrives, in order, on the replica *not* hosting the worker. With pub/sub disabled, the stream visibly stalls.
- **AC4 · Approval survives restart.** Pause at approval; restart workers; approve with an edited plan; resumed run uses plan `version=2`.
- **AC5 · Zombie safety.** `SIGSTOP` past lease TTL; reclaim; `SIGCONT`; its writes are rejected; run completes once.
- **AC6 · Fan-out distribution.** With 2 workers, researcher child runs execute on more than one worker.
- **AC7 · Loop.** The thin fixture set triggers ≥ 1 Critic loop-back and still completes within the cap.
- **AC8 · Trace.** Jaeger shows one trace spanning API → parent worker → child runs.
- **AC9 · Quality gate.** CI green: unit, integration, and `make eval` against baseline.

- **AC10 · UI.** Live view renders node states and child chips from a recorded log; approval round-trips edits; reconnect resumes from last `seq`.

KUBERNETES (KIND) — ALL SCRIPTED, AC11 AND AC12 + AC15 RUN IN CI

- **AC11 · Cluster quickstart.** `make k8s-up k8s-deploy k8s-ingest` on a fresh machine → a fixture run completes through the Ingress. All pods `Ready`, zero restarts.
- **AC12 · Pod kill.** `make k8s-demo-podkill`: `kubectl delete pod --grace-period=0 --force` on the pod owning a run → Deployment replaces the pod, another pod reclaims the run, run completes, no duplicate `findings`.
- **AC13 · Node drain.** `make k8s-demo-drain`: `kubectl drain` the node hosting the run's owner mid-run → PDBs keep ≥ 1 api and ≥ 1 worker available throughout; the run completes; the UI stream never errors.
- **AC14 · Autoscale.** `make k8s-demo-scale`: enqueue 20 runs → KEDA scales workers from 1 to ≥ 4 within 2 min; after the queue drains, back to 1 within 5 min; all 20 complete; any runs on removed pods show `run.released`, not `run.reclaimed`.
- **AC15 · Zero-downtime upgrade.** `make k8s-demo-upgrade`: `helm upgrade` to a new image tag while a run is streaming → all api and worker pods roll; the run completes; the client receives every event (reconnect allowed, gaps not); no 5xx at the Ingress.
- **AC16 · Graceful hand-off.** Deleting a worker pod *with* its normal grace period hands its run to another pod in < 5 s (vs ≈ 30 s for a forced kill) — measured and reported in `docs/kubernetes.md`.
- **AC17 · Hardened.** Namespace enforces `restricted` Pod Security with no violations; Trivy reports no critical CVEs; `helm lint` and kubeconform pass.

OUT OF SCOPE FOR V1.0 (ROADMAP)

Runs dashboard and trace view (v1.1) · gateway (P1) and policy-engine (P2) adapters (v1.2) · cross-run semantic memory (v1.3) · GitOps with Argo CD, service mesh, multi-region / multi-cluster, Temporal adapter, A2A hand-off (later).